

# BirbaBot

Andrea Venturoli 0000997951, Libera Longo 0000839018, Antonio Fiore 0000838747

Progetto di Algoritmi e Strutture Dati

A.A. 2023/2024

## Introduzione

In questo progetto il nostro scopo è costruire un algoritmo in grado di giocare a connectX, ovvero un zero-sum game con regole simili a forza4 ma con dimensioni della tabella di gioco variabili che per convenzione indicheremo con la terna M righe, N colonne e X allineamenti. Inoltre il giocatore sarà limitato da un timeout per effettuare la scelta migliore possibile.

Durante il corso abbiamo studiato alcuni algoritmi di decisione su alberi di gioco quali minimax con Alphabeta Pruning e Iterative Deepening. Il loro problema è che lo spazio di gioco è tipicamente troppo ampio da permettere la visita/generazione completa dell'albero (specie con M, N molto grandi) in tempi ragionevoli. Non potendo arrivare sempre ai nodi foglia c'è bisogno quindi di valutare le configurazioni non finali del gioco tramite un'euristica: la nostra è basata sulla "*helpfulness*", ovvero quanto una mossa in un gioco M,N,X possa aiutare sia me che l'avversario. Risolto questo problema rimaneva decidere quale algoritmo di visita usare. Dovendo tenere conto del tempo ci è sembrato naturale optare per l'Iterative Deepening, il quale esegue una visita in ampiezza senza dover fissare a priori un livello massimo. Come ulteriore ottimizzazione abbiamo pensato di memorizzare il game tree del turno precedente per rivisitarlo evitando il costoso ricalcolo dell'euristica sui nodi già visitati.

## Struttura degli algoritmi di BirbaBot

```
int selectColumn(Board)
```

Questa funzione restituisce la migliore mossa possibile entro un certo limite di tempo. Ci siamo assicurati che il nostro giocatore riesca a ritornare una mossa più o meno ottima in qualsiasi momento sfruttando la gestione delle eccezioni. La funzione è strutturata nel seguente modo:

- Viene salvato il tempo iniziale per effettuare i controlli sulla scadenza del timeout.
- Si salva l'ultima mossa fatta dall'avversario.
- Si copia la board ufficiale di gioco CXBoard B nella nostra board, su cui giocheremo per generare il game tree con Iterative Deepening.
- Se non è il nostro primo turno, la radice diventa il sottoalbero radicato nell'ultima mossa dell'avversario. Se la mossa dell'avversario non è presente nel mio vecchio game tree,

viene creata una nuova radice generando la lista di mosse ordinate in base all'euristica. Dopodiché lanciamo Iterative Deepening.

- Se è il nostro primo turno e siamo il player 1, viene giocata sempre la colonna centrale. La letteratura sui zero-sum game di questo tipo ci dice che la mossa centrale è quella che ci dà più possibilità d'azione. In questo caso, Iterative Deepening non viene chiamato.
- Se è il nostro primo turno e siamo il player 2, posizioniamo la radice nella prima mossa dell'avversario e chiamiamo Iterative Deepening.
- Infine viene ritornata la colonna associata alla cella della `bestMove`.

```
void IterativeDeepening(root, max_depth)
```

L'implementazione del nostro algoritmo Iterative Deepening è molto simile a quella studiata a lezione, a eccezione del ramo di massimizzazione Alphabeta utilizzato per tenere traccia della miglior mossa trovata durante la visita a profondità  $d$ . Quest'ultima informazione è contenuta nella variabile `bestMove_yet`, mentre `bestMove` viene aggiornata solamente quando si esegue una visita completa a profondità  $d$ . Se il timeout scade mentre valutiamo a profondità  $d$ , `bestMove` conterrà la migliore mossa individuata a profondità  $d-1$ .

```
int AlphaBeta(T, player, alpha, beta, depth)
```

L'algoritmo utilizzato è una variante di minimax con alphabeta pruning:

- Controlla se l'euristica ha trovato una configurazione immediatamente vincente o perdente.
- Se l'albero di gioco è radicato nell'ultima mossa dell'avversario, possiamo evitare di ricalcolare l'euristica visitando il figlio già generato, altrimenti creo un nuovo figlio.
- Quando raggiungiamo la profondità zero, il nodo viene valutato tramite euristica se la configurazione è non-finale, altrimenti aggiorniamo l'etichetta tenendo conto dei turni giocati:  $WIN - \text{turni\_giocati}$  se vinciamo,  $-WIN + \text{turni\_giocati}$  se perdiamo, zero in caso di pareggio.
- Se la finestra alpha-beta si chiude (e.g.:  $\beta \leq \alpha$ ) potiamo il sottomalbero.

```
void GenerateMoveList(TreeNode T)
```

Questa funzione genera tutte le mosse giocabili dal nodo  $T$  che non portano a una immediata sconfitta, aggiornando il campo del nodo dopo averle ordinate dalla migliore alla peggiore tramite la funzione di libreria `Arrays.sort()`.

```
LabeledMove[] possibleNonLosingMoves(AvailableColumns, player)
```

Metodo chiamato da `void GenerateMoveList()` per ottenere un array di mosse non immediatamente perdenti per noi o l'avversario. La funzione gestisce quattro casi:

1. Se il giocatore può vincere in una mossa, ritorna un array contenente solo quella mossa.
2. Se il giocatore può bloccare l'avversario e non può vincere in una mossa, ritorna un array contenente solo quella mossa.
3. Se il giocatore piazza un gettone sotto un allineamento vincente dell'avversario, quella mossa non viene aggiunta all'array.
4. Se la mossa non appartiene a nessuno di questi casi, viene aggiunta valutandola con la *helpfulness* calcolata da `int evaluateColumn()`.

## Euristiche

L'euristica basata sulla *helpfulness* permette di trovare una buona mossa in tempi rapidi ma non è infallibile, per questo abbiamo bisogno di visitare il game tree. Di seguito un esempio:

|   |    |    |    |    |    |   |
|---|----|----|----|----|----|---|
|   |    |    |    |    |    |   |
|   |    |    | 16 |    |    |   |
|   |    |    | O  |    |    |   |
|   |    | 18 | X  | 21 |    |   |
|   | 11 | O  | O  | X  |    |   |
| 4 | O  | X  | X  | X  | 12 | 7 |

In questa posizione (in cui deve giocare O) la mossa scelta dall'euristica sarà quella di valore 21, ma ovviamente la mossa migliore è bloccare l'avversario nella sesta colonna. Si rende quindi necessario l'uso di un algoritmo aggiuntivo in grado di evitare situazioni del genere.

```
int count(stateBoard, cell, player, area)
```

Il cuore della nostra euristica. Il suo algoritmo effettua la scansione di un'area definita attorno a una cella e ne calcola il punteggio in base al numero di configurazioni possibili e gettoni del giocatore già presenti in tali configurazioni. Sommando il punteggio di tutte le aree (verticale, orizzontale, diagonale positiva e negativa), otteniamo il valore che la cella `cell` ha per il giocatore `player`. Esistono due versioni di `count`: una che controlla in caso ci siano X allineamenti, utile per evidenziare le mosse immediatamente vincenti, e l'altra più semplice che calcola l'euristica senza preoccuparsi degli allineamenti. Sommando il punteggio calcolato dal nostro punto di vista e quello dell'avversario otteniamo la *helpfulness*.

```
int evaluateColumn(stateBoard, cell, player)
```

Questo metodo si limita a controllare che ci sia effettivamente lo spazio per calcolare l'euristica e somma il punteggio di tutte le aree, restituendo così il valore della mossa dal punto di vista di un giocatore. Anche qui abbiamo due versioni: una che controlla gli allineamenti e l'altra no. Queste differenziazioni vengono fatte poiché le configurazioni non finali sono tali proprio perché nessuno ha una immediata vittoria, quindi per ottimizzare usiamo la versione semplificata all'interno di `int EvaluateConfiguration()`.

```
int EvaluateConfiguration()
```

Assegna un punteggio a una configurazione non finale prendendo la somma della *helpfulness* dal nostro punto di vista per ogni mossa possibile e sottraendola a quella dell'avversario. Se la configurazione ha una valutazione maggiore di zero significa che favorisce BirbaBot, viceversa favorisce l'avversario. In questo metodo utilizziamo `int simpleEvaluateColumn()`, una versione della funzione creata ad hoc per velocizzare il calcolo dato che i nodi non finali vengono rivisitati spesso durante Iterative Deepening.

## Costi Computazionali

Il calcolo del costo computazionale nel caso medio è un compito difficile. Il move ordering e la visita del game tree del turno precedente permettono ad Alphabeta di avvicinarsi al caso ottimo, riuscendo a raggiungere profondità maggiori con Iterative Deepening durante i nostri test. In ogni caso calcoleremo il costo pessimo di ogni metodo del nostro algoritmo.

Nella funzione `void GenerateMoveList()` il sorting ha un costo  $n \log n$ , dove  $n$  è il numero di mosse ritornate dal metodo `void possibleNonLosingMoves()`. Il costo di quest'ultimo domina sulla crescita pseudolineare di `sort()` anche su terne  $M=100$ ,  $N=100$ ,  $X=30$ . Date queste premesse, riportiamo di seguito i costi nel caso pessimo delle funzioni principali:

Siano  $X$  il numero di allineamenti,  $C$  il numero di colonne libere,  $M$  righe,  $N$  colonne

```
int count() //  $O(4 \cdot X) = O(X)$ 
int evaluateColumn() //  $O(4 \cdot X) = O(X)$ 
int EvaluateConfiguration() //  $O(X \cdot C)$ 
LabeledMove[] possibleNonLosingMoves() //  $O(2 \cdot X \cdot C) = O(X \cdot C)$ 
void GenerateMoveList() //  $O(X \cdot C + n \log n) =$   
 //  $O(X \cdot C)$ 
int AlphaBeta() //  $O(M^{N \cdot M})$ 
void IterativeDeepening(root, max_depth) //  $O(M^{N \cdot M})$ 
```

## Conclusioni

A questo punto possiamo calcolare il costo di `int selectColumn(Board)`. Senza considerare i limiti di tempo, il costo è dominato dall'Iterative Deepening che nel caso pessimo sappiamo essere uguale a  $O(M^{N^M})$  in quanto BirbaBot è programmato per visitare sempre fino alla massima profondità (equivalente al numero di caselle ancora libere). Questo però in termini pratici non avviene quasi mai poiché l'algoritmo è interrotto dal timeout, restituendo la miglior mossa trovata nel lasso di tempo a lui concesso.

## Appendice

- Inizialmente abbiamo studiato il giocatore di [Pascal Pons](#): purtroppo il suo approccio è concentrato solo sull'ottimizzazione di un giocatore 6, 7, 4, nonostante ciò abbiamo adottato alcune delle sue scelte implementative: un sistema per anticipare le mosse perdenti e la visita con Iterative Deepening. A questo punto serviva un'euristica più generica di quella discussa da Pascal Pons in grado di valutare configurazioni non finali di una qualsiasi terna M, N, X.
- La nostra euristica implementa lo pseudo codice mostrato in questa documentazione: [“Developing a Memory Efficient Algorithm for Playing m, n, k Games”](#)
- Una delle tecniche più utilizzate per ottimizzare l'Iterative Deepening è l'aggiunta di un sistema di caching tramite Transposition Table, opzione che abbiamo preso inizialmente in considerazione usando Zobrist Hashing. Tuttavia BirbaBot gioca anche su tabelle di gioco molto grandi e risultava complesso gestire una transposition table con chiavi di dimensioni superiori a 64 bit (long size). Dopo svariati test abbiamo optato per la memorizzazione del game tree del turno precedente.

## Fonti

- Game solver di forza4 ideato da *Pascal Pons*: <http://blog.gamesolver.org/solving-connect-four/01-introduction/>
- Euristiche: [https://www.micsymposium.org/mics2016/Papers/MICS\\_2016\\_paper\\_28.pdf](https://www.micsymposium.org/mics2016/Papers/MICS_2016_paper_28.pdf)
- Pseudo codice Iterative Deepening con aggiornamento della miglior mossa alla radice: <https://gist.github.com/kartikkukreja/e58a77d6380f1af9b1f3>